Secure Restaurant Conveyor Delivery System

Model, requirements and native verification

2IMF30 System Validation, 2026–2027

Elmar Beckers, Arınç Demir, Anna Maria Petrou, Vishnu Vardhan Sirapurapu

19 October 2026

AI-use disclosure. AI assistance was used to review and revise requirements, draft and explain property formulas, prepare native verification commands and evidence records, restructure and write this report, and generate the TikZ diagram. The Boolean verification answers are computed by the mCRL2 tools.

Contents

1 System overview and purpose **2**

2 Scope, assumptions and requirements **3**

3 Actions and Data types **5**

4 MCRL2 code **10**

5 Formulas for the requirements **18**

6 Verification method and results **29**

7 Discussion and conclusion **35**

1 System overview and purpose

The system carries prepared meals from a restaurant kitchen to two tables on an indexed conveyor. A plate has an order number and a destination tag. Customers collect it from a delivery box rather than from the belt. Each box holds one plate and has an inner gate facing the conveyor, an outer gate facing the customer, and a pusher. The controller coordinates these devices so that a transfer or collection does not expose the customer to the moving belt.

An accepted order is sent to the kitchen, where a tagged plate is prepared and loaded into an empty position. The belt advances one position and stops. The kitchen and both tables receive the contents of their aligned positions. A table with a matching plate and an available box can transfer the plate, retract its pusher, close its inner gate, and open its outer gate. Otherwise it skips that visit. The kitchen and both tables report completion before movement resumes. A full box can wait for collection while its table skips later visits.

Figure 1 uses the same six positions as the checked model. The numbered nodes represent locations on the belt, rather than identities attached to plates. An advance moves each slot's contents from index i to $i + 1$ modulo six.

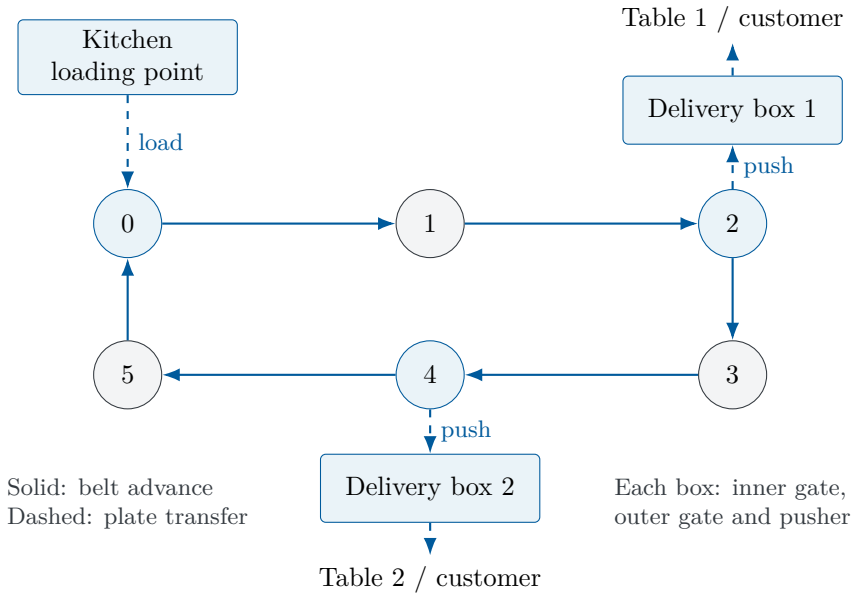


Figure 1: The six-position verification instance: kitchen at 0, Table 1 at 2, and Table 2 at 4. Gate operations and retraction are separate steps around an atomic push.

Five process instances run in parallel: an order manager, a conveyor controller, a kitchen loader, and one delivery-box controller for each table. Requests and permissions synchronize so that the order manager can block unsafe operations immediately after an emergency request. Safe finishing steps, such as retraction and gate closure, can continue until all four controlled stations can receive notification. Operator clearance then authorizes a synchronized reset. The responsibilities and complete composition appear in Section 4. Parallel composition and communication follow the process-algebra approach of Groote and Mousavi [1, Chs. 4–5].

2 Scope, assumptions and requirements

2.1 Scope and bounded instance

The product scope is one kitchen and two independently controlled, one-plate delivery boxes. Several outstanding orders and plates for the same table are allowed. A circulating belt has at most one plate per position. For verification, this is instantiated with **six belt positions, two tables, three total accepted orders, and one meal value,** `Doner`. Order numbers are 0, 1 and 2 and are not reused. The three-order limit applies over the entire execution, rather than to the number of currently outstanding orders. These are analysis bounds, not a proposed restaurant capacity policy.

Initially, the belt and boxes are empty, the belt is stopped, pushers are retracted, inner gates are open, outer gates are closed, and operation is normal. The first servicing stop must close the inner gates before movement. Although `OrderID` and `Position` are natural-number sorts, guards bound generated order numbers and the advancement counter. Station alignment is determined by fixed list indices 0, 2 and 4.

An **order** is one dish for one table and is served on one plate. A plate is **delivered** when it enters the box, and **collected** when the customer takes it through the outer gate. An **available box** is empty with its outer gate closed. A **servicing stop** begins with the synchronized round and slot snapshots while the belt is stopped. The operator's **clearance** authorizes reset; only the complete reset restores normal permissions. The model has no separate physical button-release event or second manual restart button.

Food preparation details, payments, used-plate return, temperature control, cancellation, faults in sensors or actuators, and physical timing are outside scope. Loading and delivery are atomic synchronized transitions; there is no state in which a plate is partly transferred. The model retains queued orders and plates through emergency recovery as a design choice. The suite does not separately prove complete storage preservation.

2.2 Assumptions and meaning of progress

The kitchen supplies correct plate tags and the equipment follows its commands. Routing is checked against the tag; matching that tag to the original accepted order is outside the verified scope. Sensor failures, mechanical timing, and partly completed transfers are not modeled.

Can means that a successful path exists. Such a path may include preparation, customer collection, or operator clearance. It does not promise that people will act, or that service occurs on every execution. No scheduler fairness or continued belt advancement is assumed. The progress promise in R7 is stronger: a normal stop must eventually advance unless a new emergency interrupts.

2.3 Requirements

R1. A box's two gates must never be open together.

R2. A plate may be delivered only to the table named on its tag.

R3. Each order may be loaded at most once and delivered at most once.

R4. The kitchen may load only into an empty loading position while the belt is stopped.

R5. A plate may enter only an empty delivery box.

R6. A pusher may start only with the correct plate aligned, the belt stopped, the inner gate open, and the outer gate closed. It must retract before the inner gate closes.

R7. The belt may start or advance only after every station has finished or skipped its work for the current stop, all inner gates are closed, and all pushers are retracted. The belt must eventually advance from each normal servicing stop, unless an emergency interrupts.

R8. Customers may collect only from an occupied box with its outer gate open and its inner gate closed.

R9. From an emergency request until reset, belt start or advance, loading, pushing, and inner-gate opening are blocked.

R10. Reset requires all controllers to have received the current emergency notification and the operator to have cleared that emergency.

R11. Every accepted order can be prepared and loaded during normal operation, without requiring an emergency.

R12. Every loaded plate can reach its tagged box without requiring an emergency. After delivery, collection must remain possible.

R13. Every emergency permits an approved reset. After reset, the belt can advance again without another emergency.

R14. Every reachable state must have a possible action. Waiting for a customer or operator is allowed if they can still act.

Requirements in terms of model actions

The following restates each requirement using the model's observable actions. A loading step combines `LoadPlate` and `PlateLoaded`; a delivery step combines `Push`, `BoxFilled`, and `PlateTransferred`. Each group happens as one synchronized step.

R1 `OpenInnerGate` requires the outer gate to be closed, and `OpenOuterGate` requires the inner gate to be closed. Gate positions follow from the preceding opening and closing actions.

R2 In a delivery step, the receiving table in `PlateTransferred` must match the destination carried by the plate.

R3 Neither a loading step nor a delivery step may occur twice with the same order number. Emergency and reset do not erase this history.

R4 Loading needs a fresh `LoadingPosition(Empty)` for the current stop and a stopped belt. The loading step consumes that empty-slot confirmation.

R5 After `BoxFilled` for a table, another delivery to that box is forbidden until `BoxEmptied` occurs for the same table.

R6 `ConveyorPosition` identifies the aligned plate. The `Push` step must transfer that plate with the belt stopped, the inner gate open, and the outer gate closed. Afterwards, `Retract` must precede `CloseInnerGate`.

R7 The joint `RequestPermission`/`RoundPermitted` step begins a servicing stop and supplies fresh slot information. Before movement, the kitchen and both tables must each have sent `StationDone` for that stop. `StartConveyor` and `Advance` also require closed inner gates and retracted pushers. Every continuation of the stop must eventually reach `Advance` or `StartEmergency`.

R8 `BoxEmptied` may occur only while the box contains an uncollected plate, its outer gate is open, and its inner gate is closed.

R9 From `StartEmergency` until the synchronized reset, `StartConveyor`, `Advance`, loading, `Push`, and `OpenInnerGate` are forbidden.

R10 After `StartEmergency`, all four controllers must receive the joint `EmergencyReceived` notification before `OperatorClearEmergency`. Only then may `EndEmergency` occur together with all four `EmergencyCleared` events.

R11 After each `AcceptOrder`, there must be a possible continuation containing `AddPlate`, followed by a loading step for that order and table, without `StartEmergency`.

R12 After each loading step, delivery of the same plate to its tagged table must be possible without `StartEmergency`. After delivery, `BoxEmptied` must remain possible until the plate is collected.

R13 After `StartEmergency`, a continuation through `OperatorClearEmergency` and the synchronized reset must be possible. After every reset, a continuation to `Advance` must be possible without another `StartEmergency`.

R14 After any sequence of actions, at least one further action must remain possible. Customer collection through `BoxEmptied` or clearance through `OperatorClearEmergency` also counts.

2.4 Coverage choices

The suite checks safe transport, possible service and recovery, and progress through normal stops. It does not claim complete order/plate conservation, full acceptance-to-tag identity, a full audit of skip decisions and reports, same-stop table independence, or availability of every permitted safe emergency action. The model may implement such behaviour without this report claiming it as an additional verified requirement. Historical S/F properties, the experimental R1–R22 suite, and the unregistered conditional eventual-delivery draft are outside the fourteen-property results table.

3 Actions and Data types

External interactions connect the installation with customers, kitchen staff and the operator. Physical events describe observable operations such as belt movement, gate movement and plate transfer. These observations include kitchen and staff operations; they are not limited to what a seated customer can see. Internal coordination concerns communication and control steps between the modeled components.

A label being visible in the transition system does not make it external to the installation. The model retains both physical events and selected internal coordination events so that the formulas can check the conditions for safe operation. The tables below distinguish their roles; they do not change which actions occur together.

Communication through `comm` replaces matched endpoints with a synchronized result. The `block` operator prevents unmatched endpoints from proceeding alone, while `allow` retains the specified actions and multi-actions. Hiding would remove selected action names from those labels, producing a silent step when no names remain. No hiding is used in this verification campaign. These distinctions follow the book's treatment of communication, blocking and hiding [1, Ch. 5]; the data representations follow its data definitions [1, Ch. 3].

3.1 External interactions and physical events

Table 1: External interactions and physical events

Component	Event	Meaning
Order manager	`AcceptOrder`	Accepts a meal and destination and stores a fresh order number internally. The event itself does not expose that number.
Kitchen	`AddPlate`	Prepares and associates a plate with an order number and destination.
Kitchen / belt	`LoadPlate`, `PlateLoaded`	One synchronized step authorizes loading and inserts the prepared plate into the empty loading slot.
Conveyor	`StartConveyor`, `Advance`, `StopConveyor`	Authorize movement after round completion, rotate all belt contents one indexed position, then stop. Advance carries a modulo-six movement counter.
Box	`OpenInnerGate`, `CloseInnerGate`	Open or close the conveyor-side gate of a specified table.
Box	`OpenOuterGate`, `CloseOuterGate`	Open or close the customer-side gate of a specified table.
Box / belt	`Push`, `BoxFilled`, `PlateTransferred`	One atomic transfer step moves the matching aligned plate from the belt into its destination box. This is the delivery event.
Box	`Retract`	Returns the pusher before closing the conveyor-side gate or permitting belt movement.
Customer / box	`BoxEmptied`	Represents customer collection from an occupied box with its customer-side gate open.
Order manager	`StartEmergency`	Latches the emergency request and immediately disables normal permissions, without first waiting for station notification.
Operator	`OperatorClearEmergency`	Records staff clearance. It does not release normal permissions.
Order manager	`EndEmergency`	Marks completion of emergency reset and restoration of normal permissions. It occurs together with the four controller reset events in Table 2.

3.2 Internal coordination retained for verification

These events describe the coordination protocol within the installation. They remain visible to the formulas, even though they are internal to the complete system. For example, a slot report exposes the information passed to a station, and a completion report records that station's reply. A synchronized step may combine roles: reset includes both `EndEmergency` and the four internal `EmergencyCleared` events.

Table 2: Internal coordination events retained for verification

Component	**Event**	**Meaning**
Order manager / kitchen	`OrderToKitchen`	Synchronized dispatch and receipt of an accepted order and its recorded meal and destination.
Conveyor	`RequestPermission`, `RoundPermitted`	Begin an authorized stopped servicing round, together with the loading-slot and both table-slot reports.
Conveyor / kitchen	`LoadingPosition`	Reports the actual contents of the loading slot to the kitchen.
Conveyor / box	`ConveyorPosition`	Reports a table and the contents of the belt slot aligned with it.
Kitchen	`CheckPosOne`	Marks the kitchen's check of the loading-slot information it has received.
Station	`SendSkip`	Marks a skipped kitchen or table operation. It does not itself complete the round. Despite its name, this is a retained event rather than a blocked endpoint.
Station / belt	`StationDone`	Records completion of that station's work for the current round. Successful and skipped work both require this report.
Controllers	`EmergencyReceived`	Jointly notifies the conveyor, kitchen and both boxes once all reach their modeled notification boundaries.
Kitchen	`StopLoading`	Records the kitchen's transition into emergency handling after notification.
Controllers	`EmergencyCleared`	Jointly resets all four controllers in the same step as `EndEmergency`.

3.3 Internal communication endpoints

The endpoints in Table 3 implement synchronization between components. Unmatched send, receive, request and permit endpoints are blocked; none is named directly in R1–R14. The formulas instead match the completed events, including the internal coordination events above. Normal permissions are checked in the same step as the operation, so a permission obtained before an emergency cannot be reused afterwards.

Table 3: Internal endpoints and their synchronized results

Endpoints	**Result**
`SendOrder` / `ReceiveOrder`	`OrderToKitchen`
`SendLoadingSlot` / `ReceiveLoadingSlot`	`LoadingPosition`
`SendTableSlot` / `ReceiveTableSlot`	`ConveyorPosition`
`SendPlateForLoading` / `ReceivePlateForLoading`	`PlateLoaded`, synchronized with permitted loading
`SendTableTransfer` / `ReceiveTableTransfer`	`PlateTransferred`, synchronized with pushing and filling the box
`SendStationDone` / `ReceiveStationDone`	`StationDone`
`RequestRound` / `PermitRound`	`RoundPermitted`
`RequestStartConveyor` / `PermitStartConveyor`	`StartConveyor`
`RequestAdvance` / `PermitAdvance`	`Advance`
`RequestLoadPlate` / `PermitLoadPlate`	`LoadPlate`
`RequestOpenInnerGate` / `PermitOpenInnerGate`	`OpenInnerGate`
`RequestPush` / `PermitPush`	`Push`
`SendEmergency` / `ReceiveEmergency`	`EmergencyReceived`, jointly for four controllers
`SendEmergencyReset` / `ReceiveEmergencyReset`	`EmergencyCleared`, jointly with reset

3.4 Complete synchronized steps

The symbol `|` joins actions in one transition. It differs from `||`, which composes processes in parallel. The formulas match the complete labels below, including their data, rather than treating one constituent as a separate transition.

- **Loading:** `LoadPlate(id) | PlateLoaded(plate(id,tbl))` authorizes loading and inserts the prepared plate into the belt together.
- **Delivery:** `Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl,pl)` pushes, occupies the box, and removes the matching plate from the belt together. Retraction and gate operations follow separately.
- **Round:** `RequestPermission | RoundPermitted` occurs with `LoadingPosition` and both `ConveyorPosition` snapshots. All stations therefore see the same stopped belt.
- **Notification:** four `EmergencyReceived` events jointly notify the conveyor, kitchen, and both table controllers.
- **Reset:** `EndEmergency` occurs with four `EmergencyCleared` events. Operator clearance is a separate preceding action.

3.5 Data types and storage

Table 4 describes the concrete representations in the checked source. In particular, a box stores occupancy rather than a plate identity; a plate's destination is carried on its belt-transfer event.

Table 4: Data represented in the current model

Name	Representation	Meaning
Meal	`Doner`	One representative meal kind in this instance.
TableID	`Table1`, `Table2`	Destination tables.
StationID	`Kitchen`, `ConveyorStation`, `TableStation(TableID)`	Stations addressed by coordination or emergency notification.
OrderID	Natural number	Fresh order identity; this instance accepts only 0, 1, and 2.
OrderEntry	Meal and TableID	Recorded meal and destination of an accepted order.
Plate	OrderID and TableID	Prepared plate identity and destination. The order number also identifies its single plate.
PlateSlot	Empty or Occupied(Plate)	Contents of one belt position or a buffered prepared plate.
Position	Natural number; used modulo six	Labels belt advances. Station alignment uses fixed list indices 0, 2, and 4.
Belt	List of six PlateSlot values	Live plate contents; initially empty. Each advance rotates all six values.
BoxOccupied	Boolean	Box occupancy, initially false. The box process stores occupancy rather than plate identity.
PosOneOccupied	Boolean	Kitchen observation of loading slot 0, despite the historical variable name.
GateState	Closed or Open	Both boxes initially have their conveyor-side gate open and table-side gate closed.
SystemMode	Normal or Emergency	Kitchen operating mode.
ManagerPhase	Normal, notification pending, waiting for operator, waiting for reset	Emergency latch and reset protocol of the order manager.
ConveyorPhase	Round ready, servicing, awaiting advance, emergency	Conveyor control state.
TablePhase	Waiting, opening, pushing, finishing, skipping, after reset, recovery	Delivery-box sequencing, including pusher retraction.
Order storage	OrderID to OrderEntry, with finite sets of pending and unassigned orders	Records accepted orders and work waiting for dispatch or loading.
Round replies	Finite set of StationID	Stations still required to complete the current servicing round; cancelled on emergency notification.

4 MCRL2 code

The complete checked source is Listing 1. Process choice (+) describes alternatives, action prefix (.) orders steps, and conditions enable the applicable alternatives. Parameter updates preserve omitted fields. Parallel composition interleaves independent actions and synchronizes the endpoint pairs listed in Table 3. These are the standard process-composition constructs described in the book [1, Chs. 4–5].

Process instance	Responsibility
`OrderManager`	Stores accepted orders, dispatches pending work, grants normal permissions, latches emergencies, and coordinates notification, clearance and reset.
`ConveyorBelt`	Stores six live slots, publishes fresh snapshots, waits for three completion reports, and rotates the belt before stopping.
`KitchenLoader`	Buffers orders, prepares a plate for an empty loading slot, loads it atomically, and reports completion or a skip.
`DeliveryBox(Table1)`	Controls Table 1's gate interlock, aligned transfer, retraction, collection and emergency recovery.
`DeliveryBox(Table2)`	Runs the same controller for Table 2 with its own occupancy, gates, pusher and phase.

An emergency is recorded before joint notification. Permissions for movement, loading, pushing and inner-gate opening are then unavailable. A completed transfer can still retract and close its inner gate, and notification waits for safe boundaries. Recovery closes the inner gate before opening customer access. After operator clearance, reset can retain an occupied box for later collection; an empty box closes its outer gate before reopening its inner gate. This describes the source's control flow. The specific verification claims remain R1–R14.

The listing reads the versioned model directly; its bytes match the checked input, including comments and initialization. Line numbers refer to that source file; presentation wrapping does not alter it.

Listing 1: Complete checked mCRL2 source (495 source lines).

```
% ================================================================================
% DATA TYPES
% ================================================================================
sort
    Meal = struct Doner;
    TableID = struct Table1 | Table2;
    StationID = struct TableStation(t: TableID) | Kitchen | ConveyorStation;
    OrderID = Nat;
    OrderEntry = struct order(m: Meal, t: TableID);
    Plate = struct plate(o_id: OrderID, t_id: TableID);
    BoxOccupied = Bool;
    PosOneOccupied = Bool;
    Position = Nat;
    PlateSlot = struct Empty | Occupied(p: Plate);
    GateState = struct Closed | Open;
    SystemMode = struct Normal | Emergency;
    ManagerPhase = struct ManagerNormal | NotificationPending | WaitingForOperator | WaitingForReset;
    ConveyorPhase = struct RoundReady | ServicingRound | AwaitingAdvance | ConveyorEmergency;
    TablePhase = struct WaitingForRound | OpeningForDelivery | PushingPlate |
        FinishingTransfer | SkippingRound | OpeningAfterReset | EmergencyRecovery;

% ================================================================================
% MAPS & EQUATIONS
% ================================================================================
map
    maxOrders: Nat;
    setSlot: List(PlateSlot) # Position # PlateSlot -> List(PlateSlot);
```

```
28
29 var
30     pos: Position;
31     rest: List(PlateSlot);
32     firstSlot, newSlot: PlateSlot;
33
34 eqn
35     maxOrders = 3;
36
37     % Change one slot without changing the rest of the belt.
38     setSlot([], pos, newSlot) = [];
39     setSlot(firstSlot |> rest, 0, newSlot) = newSlot |> rest;
40     pos > 0 -> setSlot(firstSlot |> rest, pos, newSlot) =
41         firstSlot |> setSlot(rest, Int2Nat(pos - 1), newSlot);
42
43 % ================================================================================
44 % ACTIONS
45 % ================================================================================
46 act
47     % Send/Receive endpoints synchronize through comm; unmatched endpoints are blocked.
48     % Request/Permit endpoints authorize an operation in the same step, never in advance.
49     % Communication results below are visible events used by the properties.
50
51     % Order manager
52     AcceptOrder: Meal # TableID; % Accept a new order for a table.
53     SendOrder: OrderID # OrderEntry; % Send an accepted order to the kitchen.
54     PermitRound; % Accept the conveyor's request for a servicing round.
55     PermitStartConveyor; % Authorize starting the belt only in normal mode.
56     PermitAdvance: Position; % Authorize advancement only in normal mode.
57     PermitLoadPlate: OrderID; % Authorize loading only in normal mode.
58     PermitOpenInnerGate: TableID; % Authorize inner-gate opening only in normal mode.
59     PermitPush: TableID; % Authorize pushing only in normal mode.
60     StartEmergency; % Latch the emergency and block new orders and unsafe operations.
61     SendEmergency: StationID; % Notify an addressed controller of the emergency.
62     OperatorClearEmergency; % Authorize reset after notification; operation resumes at EndEmergency.
63     SendEmergencyReset: StationID; % Offer synchronized reset to an addressed controller.
64     EndEmergency; % Complete all controller resets and restore normal permissions.
65
66     % Conveyor
67     RequestPermission; % Mark the start of an authorized stopped servicing round.
68     RequestRound; % Ask the order manager to authorize the round.
69     SendLoadingSlot: PlateSlot; % Send the kitchen the contents of slot 0.
70     SendTableSlot: TableID # PlateSlot; % Send a table its aligned slot.
71     ReceivePlateForLoading: Plate; % Accept atomic insertion into empty slot 0.
72     ReceiveTableTransfer: TableID # Plate; % Remove the matching plate during a table transfer.
73     ReceiveStationDone: StationID; % Accept a station's completion report.
74     RequestStartConveyor; % Request permission to start after all station reports.
75     RequestAdvance: Position; % Request permission for the next belt advancement.
76     StopConveyor; % Stop movement, including when emergency notification arrives.
77
78     % Kitchen loader
79     ReceiveOrder: OrderID # OrderEntry; % Receive and queue an order from the manager.
80     ReceiveLoadingSlot: PlateSlot; % Receive the current loading-slot contents.
81     CheckPosOne; % Check loading slot 0 using the received slot contents.
82     AddPlate: OrderID # TableID; % Prepare an order's plate before loading it.
83     RequestLoadPlate: OrderID; % Request permission to load the prepared plate.
84     SendPlateForLoading: Plate; % Offer the prepared plate for atomic belt insertion.
85     StopLoading; % Stop loading after emergency notification.
86
87     % Table controllers
88     ReceiveTableSlot: TableID # PlateSlot; % Receive the table's aligned slot.
89     RequestOpenInnerGate: TableID; % Request permission to open the inner gate.
90     CloseInnerGate: TableID; % Close the gate between the belt and delivery box.
91     OpenOuterGate: TableID; % Open the gate between the box and customer.
92     CloseOuterGate: TableID; % Close the gate between the box and customer.
93     RequestPush: TableID; % Request permission for the atomic plate transfer.
94     Retract: TableID; % Retract the pusher before closing the inner gate.
95     BoxFilled: TableID; % Mark the box occupied as part of the atomic transfer.
96     BoxEmptied: TableID; % Collect the plate through the open outer gate.
97     SendTableTransfer: TableID # Plate; % Signal the atomic box transfer and matching belt removal.
98
99     % Shared coordination
```

```
    SendSkip: StationID; % Report that a station skips its current round.
    SendStationDone: StationID; % Tell the conveyor this station has finished the round.
    ReceiveEmergency: StationID; % Receive notification at the controller's address.
    ReceiveEmergencyReset: StationID; % Accept reset after controller recovery.

    % Communication results
    OrderToKitchen: OrderID # OrderEntry; % Synchronized order dispatch and receipt.
    RoundPermitted; % Synchronized round request and manager permission.
    StartConveyor; % Synchronized permission to begin belt movement.
    Advance: Position; % Synchronized advancement; payload is the modulo-six movement counter.
    LoadPlate: OrderID; % Synchronized permission to load an order's plate.
    OpenInnerGate: TableID; % Synchronized permission to open the inner gate.
    Push: TableID; % Synchronized permission to push into the delivery box.
    LoadingPosition: PlateSlot; % Synchronized loading-slot snapshot delivery.
    ConveyorPosition: TableID # PlateSlot; % Synchronized table-slot snapshot delivery.
    PlateLoaded: Plate; % Synchronized loading and insertion into the belt.
    PlateTransferred: TableID # Plate; % Synchronized box transfer and belt removal.
    StationDone: StationID; % Synchronized station report and conveyor receipt.
    EmergencyReceived: StationID; % Synchronized addressed emergency notification.
    EmergencyCleared: StationID; % Synchronized addressed controller reset.

% ============================================================================
% PROCESSES & INITIALISATION
% ============================================================================
proc
    % Store accepted orders immediately; pending contains orders still to dispatch.
    OrderManager(
        orders: OrderID -> OrderEntry,
        nextID: OrderID,
        phase: ManagerPhase,
        pending: FSet(OrderID)
    ) =
        % Accept at most maxOrders orders (IDs 0 to maxOrders - 1).
        (phase == ManagerNormal && nextID < maxOrders) ->
            (sum meal: Meal, tbl: TableID .
                AcceptOrder(meal, tbl) .
                OrderManager(
                    orders[nextID -> order(meal, tbl)],
                    nextID + 1,
                    phase,
                    pending + {nextID}
                ))

        % Dispatch the lowest pending ID, including during an emergency.
        % A busy kitchen does not prevent accepting orders or recording an emergency.
        + sum oid: OrderID .
            (oid < maxOrders && oid in pending &&
             !(exists earlier: OrderID . earlier < oid && earlier in pending)) ->
                SendOrder(oid, orders(oid)) .
                OrderManager(pending = pending - {oid})

        % A request and its permission synchronize with the actual operation.
        % StartEmergency disables all these permissions immediately.
        + (phase == ManagerNormal) ->
            PermitRound . OrderManager()

        + (phase == ManagerNormal) ->
            PermitStartConveyor . OrderManager()

        + sum pos: Position .
            (phase == ManagerNormal && pos < 6) ->
                PermitAdvance(pos) . OrderManager()

        + sum oid: OrderID .
            (phase == ManagerNormal && oid < maxOrders) ->
                PermitLoadPlate(oid) . OrderManager()

        + sum tbl: TableID .
            (phase == ManagerNormal) ->
                (PermitOpenInnerGate(tbl) . OrderManager()
                 + PermitPush(tbl) . OrderManager())
```

```
        % Record the request immediately and disable all unsafe operations.
        + (phase == ManagerNormal) ->
            StartEmergency .
            OrderManager(phase = NotificationPending)

        % Notify all four controllers together once each reaches a safe boundary.
        % Unsafe operations are already blocked while notification waits.
        + (phase == NotificationPending) ->
            (SendEmergency(ConveyorStation) |
             SendEmergency(Kitchen) |
             SendEmergency(TableStation(Table1)) |
             SendEmergency(TableStation(Table2))) .
            OrderManager(phase = WaitingForOperator)

        % Clearance only authorizes reset; the manager remains in emergency handling.
        + (phase == WaitingForOperator) ->
            OperatorClearEmergency .
            OrderManager(phase = WaitingForReset)

        % EndEmergency restores normal permissions only with all four reset receivers.
        + (phase == WaitingForReset) ->
            (EndEmergency |
             SendEmergencyReset(ConveyorStation) |
             SendEmergencyReset(Kitchen) |
             SendEmergencyReset(TableStation(Table1)) |
             SendEmergencyReset(TableStation(Table2))) .
            OrderManager(phase = ManagerNormal);

    % belt stores the live plates; pos only labels advances with a modulo-six counter.
    ConveyorBelt(
        pos: Position,
        belt: List(PlateSlot),
        pending: FSet(StationID),
        phase: ConveyorPhase
    ) =
        % Six-slot belt: kitchen at 0, tables at 2 and 4.
        % Start a stopped round by giving all three stations fresh slot contents together.
        (phase == RoundReady) ->
            (RequestPermission | RequestRound |
             SendLoadingSlot(belt . 0) |
             SendTableSlot(Table1, belt . 2) |
             SendTableSlot(Table2, belt . 4)) .
            ConveyorBelt(
                pending = {Kitchen, TableStation(Table1), TableStation(Table2)},
                phase = ServicingRound
            )

        % Loading and insertion into empty slot 0 happen in one synchronized step.
        % Belt updates are handled one at a time.
        + sum oid: OrderID, tbl: TableID .
            (phase == ServicingRound && Kitchen in pending &&
             belt . 0 == Empty && oid < maxOrders) ->
                ReceivePlateForLoading(plate(oid, tbl)) .
                ConveyorBelt(belt = setSlot(belt, 0, Occupied(plate(oid, tbl))))

        % A push removes exactly the matching plate at that table's location.
        + sum oid: OrderID, tbl: TableID .
            (phase == ServicingRound && TableStation(tbl) in pending &&
             oid < maxOrders &&
             belt . if(tbl == Table1, 2, 4) == Occupied(plate(oid, tbl))) ->
                ReceiveTableTransfer(tbl, plate(oid, tbl)) .
                ConveyorBelt(belt = setSlot(belt, if(tbl == Table1, 2, 4), Empty))

        % Each station completes once. SendSkip alone does not clear its pending reply.
        + sum st: StationID .
            (phase == ServicingRound && st in pending) ->
                ReceiveStationDone(st) .
                ConveyorBelt(pending = pending - {st})

        % Start movement only after the kitchen and both tables have finished.
        + (phase == ServicingRound && pending == {}) ->
            RequestStartConveyor . ConveyorBelt(phase = AwaitingAdvance)
```

```
244
245         % Keep an emergency choice between starting and advancing the belt.
246         % Rotate each slot to its next index, wrapping slot 5 to 0, then serve a new round.
247         % Once advancement happens, finish stopping before accepting notification.
248         + (phase == AwaitingAdvance) ->
249             RequestAdvance(pos) . StopConveyor .
250             ConveyorBelt(
251                 pos = if(pos == 5, 0, pos + 1),
252                 belt = rhead(belt) |> rtail(belt),
253                 phase = RoundReady
254             )
255
256         % Cancel the round and its outstanding reports; retain the current belt and index.
257         + (phase != ConveyorEmergency) ->
258             ReceiveEmergency(ConveyorStation) . StopConveyor .
259             ConveyorBelt(pending = {}, phase = ConveyorEmergency)
260
261         + (phase == ConveyorEmergency) ->
262             ReceiveEmergencyReset(ConveyorStation) .
263             ConveyorBelt(phase = RoundReady);
264
265     % The inner gate faces the belt; the outer gate faces the customer.
266     % incoming is a snapshot of the slot aligned with this table for the current round.
267     DeliveryBox(
268         tableID: TableID,
269         innerGateState: GateState,
270         outerGateState: GateState,
271         boxOccupied: BoxOccupied,
272         phase: TablePhase,
273         incoming: PlateSlot
274     ) =
275         % A round starts only when this table has finished its previous work.
276         (phase == WaitingForRound) ->
277             (sum inSlot: PlateSlot .
278                 ReceiveTableSlot(tableID, inSlot) .
279                 ((inSlot == Empty || t_id(p(inSlot)) != tableID ||
280                   boxOccupied || outerGateState == Open) ->
281                     DeliveryBox(
282                         phase = SkippingRound,
283                         incoming = Empty
284                     )
285                  <>
286                     DeliveryBox(
287                         phase = if(innerGateState == Closed, OpeningForDelivery, PushingPlate),
288                         incoming = inSlot
289                     )))
290
291         % Return to a choice before each unsafe operation.
292         + (phase == OpeningForDelivery) ->
293             RequestOpenInnerGate(tableID) .
294             DeliveryBox(innerGateState = Open, phase = PushingPlate)
295
296         % Pushing, box filling, and removal from the live belt happen atomically.
297         + (phase == PushingPlate) ->
298             (RequestPush(tableID) | BoxFilled(tableID) | SendTableTransfer(tableID, p(incoming))) .
299             DeliveryBox(boxOccupied = true, phase = FinishingTransfer, incoming = Empty)
300
301         % StartEmergency may occur during this sequence; table notification waits for it to finish.
302         % Retract before closing the inner gate and opening the outer gate.
303         + (phase == FinishingTransfer) ->
304             Retract(tableID) . CloseInnerGate(tableID) . OpenOuterGate(tableID) .
305             SendStationDone(TableStation(tableID)) .
306             DeliveryBox(innerGateState = Closed, outerGateState = Open, phase = WaitingForRound)
307
308         % Even an empty box must close its inner gate before movement.
309         % Report a skip, then completion; only completion releases the conveyor.
310         + (phase == SkippingRound) ->
311             ((innerGateState == Open) ->
312                 CloseInnerGate(tableID) .
313                 SendSkip(TableStation(tableID)) . SendStationDone(TableStation(tableID)) .
314                 DeliveryBox(innerGateState = Closed, phase = WaitingForRound)
315              <>
```

```
316                 SendSkip(TableStation(tableID)) . SendStationDone(TableStation(tableID)) .
317                 DeliveryBox(phase = WaitingForRound))
318
319         % After collection, finish closing before accepting a new round or notification.
320         + (phase == WaitingForRound && boxOccupied && outerGateState == Open) ->
321             BoxEmptied(tableID) . CloseOuterGate(tableID) .
322             DeliveryBox(outerGateState = Closed, boxOccupied = false)
323
324         % A second emergency can block inner-gate opening immediately after reset.
325         + (phase == OpeningAfterReset) ->
326             RequestOpenInnerGate(tableID) .
327             DeliveryBox(innerGateState = Open, phase = WaitingForRound)
328
329         % Cancel unfinished work at a safe boundary and discard only the slot snapshot.
330         % A cancelled push leaves the plate on the belt; a completed push leaves the box occupied.
331         + (phase == WaitingForRound || phase == OpeningForDelivery ||
332            phase == PushingPlate || phase == OpeningAfterReset) ->
333             ReceiveEmergency(TableStation(tableID)) .
334             DeliveryBox(phase = EmergencyRecovery, incoming = Empty)
335
336         % Recovery preserves occupancy and closes the belt-side gate before customer access.
337         + (phase == EmergencyRecovery && innerGateState == Open) ->
338             CloseInnerGate(tableID) . OpenOuterGate(tableID) .
339             DeliveryBox(innerGateState = Closed, outerGateState = Open)
340
341         + (phase == EmergencyRecovery && innerGateState == Closed && outerGateState == Closed) ->
342             OpenOuterGate(tableID) .
343             DeliveryBox(outerGateState = Open)
344
345         + (phase == EmergencyRecovery && innerGateState == Closed && outerGateState == Open &&
    boxOccupied) ->
346             BoxEmptied(tableID) .
347             DeliveryBox(boxOccupied = false)
348
349         % Reset keeps an occupied box accessible. An empty box restores inner-open readiness
350         % by closing the outer gate first, then requesting permission to open the inner gate.
351         + (phase == EmergencyRecovery && innerGateState == Closed && outerGateState == Open) ->
352             ReceiveEmergencyReset(TableStation(tableID)) .
353             (boxOccupied ->
354                 DeliveryBox(phase = WaitingForRound)
355              <>
356                 CloseOuterGate(tableID) .
357                 DeliveryBox(outerGateState = Closed, phase = OpeningAfterReset));
358
359     % waitingForRound is true while awaiting fresh slot contents, after loading,
360     % skipping, or cancellation. unassigned contains orders still waiting to be loaded.
361     KitchenLoader(
362         orders: OrderID -> OrderEntry,
363         unassigned: FSet(OrderID),
364         posOneOccupied: PosOneOccupied,
365         waitingForRound: Bool,
366         mode: SystemMode,
367         prepared: PlateSlot
368     ) =
369         % Buffer incoming orders whenever ready to receive them, including during emergencies.
370         sum oid: OrderID, entry: OrderEntry .
371             (oid < maxOrders && (prepared == Empty || waitingForRound)) ->
372                 ReceiveOrder(oid, entry) .
373                 KitchenLoader(
374                     orders[oid -> entry],
375                     unassigned + {oid},
376                     posOneOccupied,
377                     waitingForRound,
378                     mode,
379                     prepared
380                 )
381
382         % Fresh loading-slot contents start one round: perform one load or one skip.
383         + (mode == Normal && waitingForRound) ->
384             (sum inSlot: PlateSlot .
385                 ReceiveLoadingSlot(inSlot) . CheckPosOne .
386                 KitchenLoader(posOneOccupied = inSlot != Empty, waitingForRound = false))
```

```
387
388         % Skipping still requires a completion report before the belt can move.
389         + (mode == Normal && !waitingForRound && (posOneOccupied || unassigned == {})) ->
390             SendSkip(Kitchen) . SendStationDone(Kitchen) .
391             KitchenLoader(waitingForRound = true)
392
393         % Choose any queued order; preparation order is unrestricted.
394         + sum oid: OrderID .
395             (mode == Normal && oid in unassigned && !posOneOccupied &&
396              !waitingForRound && prepared == Empty && oid < maxOrders) ->
397                AddPlate(oid, t(orders(oid))) .
398                KitchenLoader(prepared = Occupied(plate(oid, t(orders(oid)))))
399
400         % Load and insert atomically, then finish reporting before accepting notification.
401         % Keep the prepared plate until loading succeeds, including across emergencies.
402         + (mode == Normal && !waitingForRound && !posOneOccupied && prepared != Empty) ->
403             (RequestLoadPlate(o_id(p(prepared))) | SendPlateForLoading(p(prepared))) .
404             SendStationDone(Kitchen) .
405             KitchenLoader(unassigned = unassigned - {o_id(p(prepared))}, waitingForRound = true,
    prepared = Empty)
406
407         % Cancel this round while retaining queued orders and any prepared plate.
408         + (mode == Normal) ->
409             ReceiveEmergency(Kitchen) . StopLoading .
410             KitchenLoader(mode = Emergency, waitingForRound = true)
411
412         + (mode == Emergency) ->
413             ReceiveEmergencyReset(Kitchen) .
414             KitchenLoader(mode = Normal);
415
416     System =
417         % Internal sends, receives, requests, and permissions must synchronize through comm.
418         % Blocking unmatched endpoints prevents either side from proceeding alone.
419         block(
420             {SendTableSlot, ReceiveTableSlot,
421              SendOrder, ReceiveOrder,
422              SendEmergency, ReceiveEmergency,
423              SendEmergencyReset, ReceiveEmergencyReset,
424              RequestRound, PermitRound,
425              RequestStartConveyor, PermitStartConveyor,
426              RequestAdvance, PermitAdvance,
427              RequestLoadPlate, PermitLoadPlate,
428              RequestOpenInnerGate, PermitOpenInnerGate,
429              RequestPush, PermitPush,
430              SendLoadingSlot, ReceiveLoadingSlot,
431              SendPlateForLoading, ReceivePlateForLoading,
432              SendTableTransfer, ReceiveTableTransfer,
433              SendStationDone, ReceiveStationDone},
434             comm(
435                 {
436                     SendTableSlot | ReceiveTableSlot -> ConveyorPosition,
437                     SendEmergency | ReceiveEmergency -> EmergencyReceived,
438                     SendEmergencyReset | ReceiveEmergencyReset -> EmergencyCleared,
439                     RequestRound | PermitRound -> RoundPermitted,
440                     RequestStartConveyor | PermitStartConveyor -> StartConveyor,
441                     RequestAdvance | PermitAdvance -> Advance,
442                     RequestLoadPlate | PermitLoadPlate -> LoadPlate,
443                     RequestOpenInnerGate | PermitOpenInnerGate -> OpenInnerGate,
444                     RequestPush | PermitPush -> Push,
445                     SendLoadingSlot | ReceiveLoadingSlot -> LoadingPosition,
446                     SendPlateForLoading | ReceivePlateForLoading -> PlateLoaded,
447                     SendTableTransfer | ReceiveTableTransfer -> PlateTransferred,
448                     SendStationDone | ReceiveStationDone -> StationDone,
449                     SendOrder | ReceiveOrder -> OrderToKitchen
450                 },
451                 OrderManager(
452                     lambda id: OrderID . order(Doner, Table1),
453                     0,
454                     ManagerNormal,
455                     {}
456                 )
457                 || ConveyorBelt(
```

```
                    0,
                    [Empty, Empty, Empty, Empty, Empty, Empty],
                    {},
                    RoundReady
                )
                || KitchenLoader(
                    lambda oid: OrderID . order(Doner, Table1),
                    {},
                    false,
                    true,
                    Normal,
                    Empty
                )
                || DeliveryBox(Table1, Open, Closed, false, WaitingForRound, Empty)
                || DeliveryBox(Table2, Open, Closed, false, WaitingForRound, Empty)
            )
        );

% Keep communication results visible and retain the required atomic multi-actions.
% Independent actions interleave; unrelated actions cannot combine into extra multi-actions.
init allow(
    {
        AcceptOrder, AddPlate, StopLoading,
        StartEmergency, OperatorClearEmergency,
        StartConveyor, Advance, StopConveyor,
        OpenInnerGate, CloseInnerGate, OpenOuterGate, CloseOuterGate,
        Retract, OrderToKitchen, CheckPosOne, BoxEmptied, SendSkip, StationDone,
        LoadPlate | PlateLoaded,
        Push | BoxFilled | PlateTransferred,
        RequestPermission | RoundPermitted | LoadingPosition |
            ConveyorPosition | ConveyorPosition,
        EmergencyReceived | EmergencyReceived |
            EmergencyReceived | EmergencyReceived,
        EndEmergency | EmergencyCleared | EmergencyCleared |
            EmergencyCleared | EmergencyCleared
    },
    System
);
```

5 Formulas for the requirements

5.1 Reading the formulas

The formulas use the first-order modal μ-calculus [1, Ch. 6]. `[r] P` requires `P` after every finite path matching regular expression `r`; `<r> P` requires one such path. In path expressions, `.` concatenates, `*` allows zero or more repetitions, `!` negates an action condition, and `||` is action-condition disjunction. `true` matches any transition. Data quantifiers choose values, while `val(...)` tests a Boolean data expression. `&&` conjoins obligations.

For example, `[true* . A . (!B)* . C] false` forbids `C` after `A` until `B` occurs. In contrast, `[true* . A] <r . B> true` requires a possible continuation to `B` after each reachable `A`. It does not require that continuation to be selected.

A greatest fixed point, `nu`, maintains an invariant while updating a small local memory. The monitors below store an empty-slot confirmation, an aligned slot, a station-completion stage, a reset phase, or the next accepted order number. They do not store complete lifecycle histories. A least fixed point, `mu`, expresses finite progress: R7's `mu Finish` requires advancement or an emergency on every continuation from a normal stop. Its `<true> true` conjunct also rejects a deadlock before that goal.

Each following listing reads its complete versioned `.mcf` file, matching the checked revision. R12 uses the separately recorded, user-approved revision with an explicit order-ID guard described in Section 6; the other thirteen formulas are unchanged from the initial snapshot. The `CHECK` comments name conjuncts for explanation; each file is submitted to `lts2pbes` as one formula. No conjuncts are checked as separate jobs in this campaign. The pre-run comment in R7 saying it was not yet checked records the input's state when frozen; the fresh outcome is in Section 6.

5.2 Observations used by the properties

The suite checks observable system behaviour together with selected internal coordination conditions, using the distinction in Section 3. R4 uses the round's `LoadingPosition` report to confirm an empty loading slot; R6 uses `ConveyorPosition` to identify the aligned plate. Both match the complete round label and invalidate their snapshots on advancement or the complete reset. R7 additionally observes `StationDone` to check that all three reporting stations have completed the current stop before movement.

R9 and R13 identify reset by the complete label containing `EndEmergency` and all four `EmergencyCleared` events. R10 also observes the joint `EmergencyReceived` notification to check notification, operator clearance and reset in order. These formulas therefore include protocol observations rather than using only external interactions. None names the individual blocked communication endpoints. R14 names no particular event: its `true` matches every model transition, including internal coordination, so it checks whether some action remains possible rather than whether an external action is immediately possible.

5.3 R1: Gate interlock

Requirement R1: The first conjunct handles the initially open inner gate. The other two forbid opening one gate after the other has opened and before it has closed. Both directions are necessary.

Listing 2: Complete formula for R1.

```
% R1: A box's two gates must never be open together.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: initial-inner-gate
% The inner gate starts open: close it before opening the outer gate.
(
forall tbl: TableID .
[(!CloseInnerGate(tbl))* . OpenOuterGate(tbl)] false
)
% END CHECK
&&
% CHECK: inner-to-outer
% After opening the inner gate, forbid outer opening until inner closure.
(
forall tbl: TableID .
[true* . OpenInnerGate(tbl) . (!CloseInnerGate(tbl))* . OpenOuterGate(tbl)] false
)
% END CHECK
&&
% CHECK: outer-to-inner
% After opening the outer gate, forbid inner opening until outer closure.
(
forall tbl: TableID .
[true* . OpenOuterGate(tbl) . (!CloseOuterGate(tbl))* . OpenInnerGate(tbl)] false
)
% END CHECK
```

5.4 R2: Tagged destination

Requirement R2: A forbidden delivery is a complete transfer whose receiving table differs from the plate tag. Correct tagging by the kitchen remains an assumption.

Listing 3: Complete formula for R2.

```
% R2: A plate may be delivered only to the table named on its tag.
% Scope: six belt positions, two tables, three total orders; atomic events.
% Assumption: the kitchen supplies correct plate tags; acceptance-to-tag matching is outside this
   check.

% CHECK: tag
% Reject a delivery whose receiving table differs from its plate tag.
(
[true* . (exists tbl: TableID, pl: Plate .
    val(tbl != t_id(pl)) && (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl)))] false
)
% END CHECK
```

5.5 R3: At-most-once processing

Requirement R3: For each admitted order number, the first conjunct forbids a second load and the second forbids a second delivery. Neither history is cleared by an emergency or reset.

Listing 4: Complete formula for R3.

```
% R3: Each order may be loaded at most once and delivered at most once.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: one-load
% For each admitted order number, forbid a second load.
(
forall oid: OrderID . (val(oid < maxOrders) =>
[true* . (exists pl: Plate . (LoadPlate(oid) | PlateLoaded(pl))) . true* .
         (exists pl: Plate . (LoadPlate(oid) | PlateLoaded(pl)))] false)
)
% END CHECK
&&
% CHECK: one-delivery
% Forbid a second delivery of the same order, even with a different tag.
(
forall oid: OrderID . (val(oid < maxOrders) =>
[true* . (exists at, tag: TableID . (Push(at) | BoxFilled(at) | PlateTransferred(at, plate(oid, tag)))
  ) . true* .
         (exists at, tag: TableID . (Push(at) | BoxFilled(at) | PlateTransferred(at, plate(oid, tag)))
  )] false)
)
% END CHECK
```

5.6 R4: Safe loading

Requirement R4: The Boolean monitor remembers whether the current round supplied an empty loading slot. A load consumes that confirmation; advance and reset invalidate it. The second conjunct independently forbids loading between movement and the next stop.

Listing 5: Complete formula for R4.

```
% R4: The kitchen may load only into an empty loading position while the belt is stopped.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: empty-confirmation
% Remember only the loading-slot check for this stop; loading consumes that confirmation.
(
nu EmptySlot(empty: Bool = false) .
(
    forall k, a, b: PlateSlot . [(RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b))] EmptySlot(k == Empty)
    && [(exists oid: OrderID, pl: Plate . (LoadPlate(oid) | PlateLoaded(pl)))] (val(empty) && EmptySlot(false))
    && [((exists pos: Position . Advance(pos)) || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))] EmptySlot(false)
    && [!((exists k, a, b: PlateSlot . (RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b))) || (exists oid: OrderID, pl: Plate . (LoadPlate(oid) | PlateLoaded(pl))) || (exists pos: Position . Advance(pos)) || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))] EmptySlot(empty)
)
)
% END CHECK
&&
% CHECK: stopped
% A load cannot occur between movement and the next stop.
(
[true* . (StartConveyor || (exists pos: Position . Advance(pos))) . (!StopConveyor)* . (exists oid: OrderID, pl: Plate . (LoadPlate(oid) | PlateLoaded(pl)))] false
)
% END CHECK
```

5.7 R5: Empty delivery box

Requirement R5: After delivery to a box, a second transfer to that box is forbidden until collection. This uses the initially empty boxes and the model's atomic filling and collection events.

Listing 6: Complete formula for R5.

```
% R5: A plate may enter only an empty delivery box.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: empty-box
% Between a delivery and collection, another delivery to that box is forbidden.
(
forall tbl: TableID .
[true* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) . (!BoxEmptied(tbl))* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl)))] false
)
% END CHECK
```

5.8 R6: Safe pushing and retraction

Requirement R6: The aligned-slot monitor records the current round's snapshot for one table. A push must consume exactly that plate and match its tag. The remaining conjuncts require a stopped belt, an open inner gate, a closed outer gate, and retraction before inner-gate closure.

Listing 7: Complete formula for R6.

```
% R6: A pusher may start only with the correct plate aligned, the belt stopped, the inner gate open, and the outer gate closed. It must retract before the inner gate closes.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: alignment
% Remember one current aligned slot; a transfer must consume that same plate.
(
forall tbl: TableID .
nu Aligned(slot: PlateSlot = Empty) .
(
    forall k, a, b: PlateSlot . [(RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b))]
        Aligned(if(tbl == Table1, a, b))
 && forall pl: Plate . [(Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))]
        (val(slot == Occupied(pl) && t_id(pl) == tbl) && Aligned(Empty))
 && [((exists pos: Position . Advance(pos)) || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))] Aligned(Empty)
 && [!((exists k, a, b: PlateSlot . (RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b))) || (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) || (exists pos: Position . Advance(pos)) || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))] Aligned(slot)
)
)
% END CHECK
&&
% CHECK: stopped
% Pushing is forbidden between belt movement and stopping.
(
[true* . (StartConveyor || (exists pos: Position . Advance(pos))) . (!StopConveyor)* . (exists tbl: TableID, pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl)))] false
)
% END CHECK
&&
% CHECK: inner-open
% The inner gate starts open; after closure it must reopen before pushing.
(
forall tbl: TableID .
[true* . CloseInnerGate(tbl) . (!OpenInnerGate(tbl))* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl)))] false
)
% END CHECK
&&
% CHECK: outer-closed
% After outer opening, close it before pushing.
(
forall tbl: TableID .
[true* . OpenOuterGate(tbl) . (!CloseOuterGate(tbl))* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl)))] false
)
% END CHECK
&&
% CHECK: retract
% After pushing, retract before closing the inner gate.
(
forall tbl: TableID .
[true* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) . (!Retract(tbl))* . CloseInnerGate(tbl)] false
)
% END CHECK
```

5.9 R7: Movement barrier and stop progress

Requirement R7: For each reporting station, stage 0 means no current stop, stage 1 means awaiting its completion, and stage 2 means completed. A fresh round starts stage 1; StationDone completes it. Start and advance require stage 2. Emergency, reset and advancement clear the old stop. Further conjuncts check gate closure and retraction. The final least fixed point forbids endless postponement of advancement during an uninterrupted normal stop, including deadlock. This does not audit every skip decision or duplicate report.

Listing 8: Complete formula for R7.

```
% R7: The belt may start or advance only after every station has finished or skipped its work for the current stop, all inner gates are closed, and all pushers are retracted. The belt must eventually advance from each normal servicing stop, unless an emergency interrupts.
% Scope: six belt positions, two tables, three total orders; atomic events.
% Revised station-completion clause: not yet model-checked on the full model.

% CHECK: station-completion
% Check the kitchen and both tables separately; ConveyorStation does not report.
% Stage 0: no current stop; 1: awaiting this station; 2: station completed.
% Successful work and a skip both end with StationDone. SendSkip alone is not
% the completion acknowledgement. Reports outside a current stop are ignored.
% A new stop clears earlier reports. Emergency/reset cancel the old stop.
(
forall st: StationID . (val(st != ConveyorStation) =>
nu Completion(stage: Nat = 0) .
(
    [(exists k, a, b: PlateSlot . (RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b)))]
        Completion(1)
    && [StationDone(st)] Completion(if(stage == 1, 2, stage))
    && [StartConveyor] (val(stage == 2) && Completion(stage))
    && [(exists pos: Position . Advance(pos))]
        (val(stage == 2) && Completion(0))
    && [(StartEmergency || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))]
        Completion(0)
    && [!((exists k, a, b: PlateSlot . (RequestPermission | RoundPermitted | LoadingPosition(k) | ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b))) || StationDone(st) || StartConveyor || (exists pos: Position . Advance(pos)) || StartEmergency || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))]
        Completion(stage)
)
)
)
% END CHECK
&&

% CHECK: initial-closure
% Initially open inner gates must close before movement.
(
forall tbl: TableID .
[(!CloseInnerGate(tbl))* . (StartConveyor || (exists pos: Position . Advance(pos)))] false
)
% END CHECK
&&
% CHECK: closed-gates
% Every later inner opening must be followed by closure before movement.
(
forall tbl: TableID .
[true* . OpenInnerGate(tbl) . (!CloseInnerGate(tbl))* . (StartConveyor || (exists pos: Position . Advance(pos)))] false
)
% END CHECK
&&
% CHECK: retracted
% A pusher must retract before belt movement.
(
forall tbl: TableID .
```

```
[true* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) . (!Retract(tbl
  ))* . (StartConveyor || (exists pos: Position . Advance(pos)))] false
)
% END CHECK
&&
% CHECK: finish-stop
% On every continuation, the stop reaches advancement or an emergency; deadlock is a failure.
(
[true* . (exists k, a, b: PlateSlot . (RequestPermission | RoundPermitted | LoadingPosition(k) |
  ConveyorPosition(Table1, a) | ConveyorPosition(Table2, b)))]
mu Finish .
(
    <true> true
    && [!((exists pos: Position . Advance(pos)) || StartEmergency)] Finish
)
)
% END CHECK
```

5.10 R8: Safe collection

Requirement R8: The first pair of conjuncts checks initial and later occupancy. The next pair checks the initially closed and subsequently closed outer gate. The last pair checks the initially open and subsequently opened inner gate. Collection therefore requires an uncollected delivery and safe gate positions.

Listing 9: Complete formula for R8.

```
% R8: Customers may collect only from an occupied box with its outer gate open and its inner gate closed.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: initial-occupancy
% Initially empty boxes cannot be collected before a delivery.
(
forall tbl: TableID .
[(!(exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))))* . BoxEmptied(tbl)] false
)
% END CHECK
&&
% CHECK: occupied
% After collection there must be another delivery before the next collection.
(
forall tbl: TableID .
[true* . BoxEmptied(tbl) . (!(exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))))* . BoxEmptied(tbl)] false
)
% END CHECK
&&
% CHECK: initial-outer
% Initially closed outer gates must open before collection.
(
forall tbl: TableID .
[(!OpenOuterGate(tbl))* . BoxEmptied(tbl)] false
)
% END CHECK
&&
% CHECK: outer-open
% After outer closure, it must reopen before collection.
(
forall tbl: TableID .
[true* . CloseOuterGate(tbl) . (!OpenOuterGate(tbl))* . BoxEmptied(tbl)] false
)
% END CHECK
&&
% CHECK: initial-inner
% Initially open inner gates must close before collection.
(
forall tbl: TableID .
[(!CloseInnerGate(tbl))* . BoxEmptied(tbl)] false
)
% END CHECK
&&
% CHECK: inner-closed
% After inner opening, it must close before collection.
(
forall tbl: TableID .
[true* . OpenInnerGate(tbl) . (!CloseInnerGate(tbl))* . BoxEmptied(tbl)] false
)
% END CHECK
```

5.11 R9: Emergency inhibition

Requirement R9: Between the request and the complete reset, the listed unsafe transitions are forbidden. Safe finishing actions are not forbidden by this formula; it does not separately prove that every safe action remains possible.

Listing 10: Complete formula for R9.

```
% R9: From an emergency request until reset, belt start or advance, loading, pushing, and inner-gate opening are blocked.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: inhibition
% Forbid these unsafe events after the request until the complete reset.
(
[true* . StartEmergency .
 (!(EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))* .
 (StartConveyor || (exists pos: Position . Advance(pos)) || (exists oid: OrderID, pl: Plate . (LoadPlate(oid) | PlateLoaded(pl))) || (exists tbl: TableID, pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) || (exists tbl: TableID . OpenInnerGate(tbl)))] false
)
% END CHECK
```

5.12 R10: Approved reset

Requirement R10: One four-stage monitor checks normal operation, request, complete notification, and operator clearance. Only the cleared phase permits the complete reset. Other actions retain the phase. Physical repairs and a separate button protocol are outside this check.

Listing 11: Complete formula for R10.

```
% R10: Reset requires all controllers to have received the current emergency notification and the operator to have cleared that emergency.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: reset-phase
% One four-state monitor: 0 normal, 1 requested, 2 all notified, 3 cleared.
(
nu ResetPhase(stage: Nat = 0) .
(
    [StartEmergency] (val(stage == 0) && ResetPhase(1))
    && [(EmergencyReceived(ConveyorStation) | EmergencyReceived(Kitchen) | EmergencyReceived(TableStation(Table1)) | EmergencyReceived(TableStation(Table2)))] (val(stage == 1) && ResetPhase(2))
    && [OperatorClearEmergency] (val(stage == 2) && ResetPhase(3))
    && [(EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2)))] (val(stage == 3) && ResetPhase(0))
    && [!(StartEmergency || (EmergencyReceived(ConveyorStation) | EmergencyReceived(Kitchen) | EmergencyReceived(TableStation(Table1)) | EmergencyReceived(TableStation(Table2))) || OperatorClearEmergency || (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2))))]
      ResetPhase(stage)
)
)
% END CHECK
```

5.13 R11: Possible accepted-order loading

Requirement R11: AcceptOrder does not expose its assigned number. The nextID counter reconstructs the model's acceptance sequence from zero. After each acceptance, a diamond requires preparation and loading of that order and table along a path without StartEmergency.

Listing 12: Complete formula for R11.

```
% R11: Every accepted order can be prepared and loaded during normal operation, without requiring an emergency.
% Scope: six belt positions, two tables, three total orders; atomic events.
% Capability: a successful path exists; this does not promise service on every execution.

% CHECK: load-capability
% AcceptOrder hides its ID; nextID counts acceptances from zero. The diamond asks for one normal preparation/loading path.
(
nu Orders(nextID: OrderID = 0) .
(
    forall meal: Meal, tbl: TableID . [AcceptOrder(meal, tbl)]
    (
        <(!StartEmergency)* . AddPlate(nextID, tbl) .
         (!StartEmergency)* . (LoadPlate(nextID) | PlateLoaded(plate(nextID, tbl)))> true
        && Orders(nextID + 1)
    )
    && [!(exists meal: Meal, tbl: TableID . AcceptOrder(meal, tbl))] Orders(nextID)
)
)
% END CHECK
```

5.14 R12: Possible delivery and continued collection access

Requirement R12: After every actual load, the first conjunct requires a path to delivery of that same tagged plate without a new emergency. Collection of other plates may be used along that path. The order-ID guard makes the existing three-order bound explicit: other IDs cannot trigger a load in this model. After each delivery, the second conjunct requires collection to remain reachable from every state before collection, including states reached during emergencies.

Listing 13: Complete formula for R12.

```
% R12: Every loaded plate can reach its tagged box without requiring an emergency. After delivery, collection must remain possible.
% Scope: six belt positions, two tables, three total orders; atomic events.
% Capability: a successful path exists; this does not promise service on every execution.

% CHECK: delivery-capability
% Only IDs below maxOrders can be loaded in this bounded model.
% For every actual load, exhibit a path to its tagged box without a new emergency. Collection of other plates may occur.
(
forall oid: OrderID, tbl: TableID . (val(oid < maxOrders) =>
[true* . (LoadPlate(oid) | PlateLoaded(plate(oid, tbl)))]
<(!StartEmergency)* . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, plate(oid, tbl)))> true
)
)
% END CHECK
&&
% CHECK: collection-capability
% At every state after delivery and before collection, collection is still reachable. Operator clearance and reset may occur.
(
forall tbl: TableID .
[true* . (exists pl: Plate . (Push(tbl) | BoxFilled(tbl) | PlateTransferred(tbl, pl))) . (!BoxEmptied(tbl))*]
<true* . BoxEmptied(tbl)> true
)
% END CHECK
```

5.15 R13: Possible emergency recovery

Requirement R13: The first conjunct requires a reset path after every request; R10 supplies its notification and clearance ordering. The second requires a path to advancement after every reset without another emergency. Both paths exclude customer collection. This is stronger than the English requirement, which does not require collection independence.

Listing 14: Complete formula for R13.

```
% R13: Every emergency permits an approved reset. After reset, the belt can advance again without
   another emergency.
% Scope: six belt positions, two tables, three total orders; atomic events.
% Capability: a successful path exists; this does not promise service on every execution.
% The unchanged formula additionally checks that collection is not required;
% this is stronger than the revised English requirement.

% CHECK: recover
% After every request, a complete reset is reachable without customer collection.
(
[true* . StartEmergency]
<(!(exists tbl: TableID . BoxEmptied(tbl)))* . (EndEmergency | EmergencyCleared(ConveyorStation) |
   EmergencyCleared(Kitchen) | EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(
   Table2)))> true
)
% END CHECK
&&
% CHECK: resume
% After every reset, normal advancement is reachable without another emergency or collection.
(
[true* . (EndEmergency | EmergencyCleared(ConveyorStation) | EmergencyCleared(Kitchen) |
   EmergencyCleared(TableStation(Table1)) | EmergencyCleared(TableStation(Table2)))]
<(!(StartEmergency || (exists tbl: TableID . BoxEmptied(tbl))))* . (exists pos: Position . Advance(pos
   ))> true
)
% END CHECK
```

5.16 R14: No deadlock

Requirement R14: After every finite execution, some transition is enabled. Internal coordination, customer collection and operator clearance all count. This forbids terminal states. It does not require an external action to be immediately possible or guarantee eventual delivery or human participation.

Listing 15: Complete formula for R14.

```
% R14: Every reachable state must have a possible action. Waiting for a customer or operator is
   allowed if they can still act.
% Scope: six belt positions, two tables, three total orders; atomic events.

% CHECK: deadlock
% Every reachable state has at least one outgoing transition.
(
[true*] <true> true
)
% END CHECK
```

6 Verification method and results

6.1 Environment and frozen revision

This campaign starts from the current working-tree files rather than substituting results from an earlier campaign. The repository's base commit is

`42dd9c310e4e672fe21237ec6794c5372cf12917`

The working tree was dirty: catalog, README and project configuration changes were tracked, and the current R files and supporting material were untracked. Commit identity alone therefore does not identify the checked inputs. The compact evidence under `verification/` records the original status, tracked diff, input hashes and native results. Large generated graphs and PBES files are omitted from Git and can be regenerated using the recorded commands. R12 has one separately frozen, user-approved revision; Table 5 gives the selected report inputs, including that revision.

The toolset is **mCRL2 202607.0 (Release)**, installed at `/Applications/mCRL2.app/Contents/bin`. The host runs **macOS 27.0.1, build 26A434; Darwin 27.0.0; arm64**. Hardware is **Apple M5, Mac17,4, 10 physical and logical CPU cores, 24 GiB memory**. Tools run sequentially with their default thread settings. The recorded host command output and installed tool help are retained under `verification/metadata/`. Native compiled rewriting uses the installed toolchain; compilation time is included in the relevant stage.

The following working directories describe the recorded run on the verifying machine; teammates can use any fresh folder with the same relative input layout. The compact evidence is described in `verification/README.md`. The original campaign directory, relative to the project repository, is `verification-report-20261019T150039Z`. Its `properties/` contains the frozen formulas, its top level contains the frozen model, and commands run in its `run/` subdirectory. The exact absolute working directory is:

```
/Users/arinc/Projects/2IMF30-System-Validation/restaurant-conveyor-delivery-s
ystem/verification-report-20261019T150039Z/run
```

The report reads the versioned model and formulas directly from the repository. Their hashes match the selected verification inputs recorded in `verification/inputs.sha256`. The model, all English requirements, and thirteen formulas are unchanged.

The original R12 attempt was interrupted at the user's request without a Boolean answer. Its revised first conjunct adds `val(oid < maxOrders) =>`, as R3 already does. Loading in the model requires that same bound, so IDs outside it have no triggering load and their original obligations are vacuously true. This preserves R12's meaning on this bounded model. Both versions use the same three-order model; the guard restricts the relevant formula cases, rather than reducing the model's order bound. Section 6.5 explains the subsequent diagnostic evidence.

The revised R12 uses the same four commands from this separate working directory:

```
/Users/arinc/Projects/2IMF30-System-Validation/restaurant-conveyor-delivery-s
ystem/verification-report-20261019T150039Z/revisions/R12-bounded-20261019T1814
16Z/run
```

Its `reduced.lts` links to the freshly generated graph in the original campaign. Recorded SHA-256 hashes confirm the identical model and graph. The revision runs after R13 and R14; it does not overlap another property check.

Table 5: SHA-256 hashes of the selected verification inputs; R12 uses its explicit order-ID guard.

Input	SHA-256 (two consecutive halves)
Model	`145cc14e4efcd065b7cd15068c8b0bc8` `bcdaecc198070f6e4d68b8cfe5a90fae`
R1	`f40f80622b4e11df184af4bbad315df7` `5e334e092c3bda68afc36978f18e4dbd`
R2	`347cf50e693fdac727e37ef27f0bf869` `122d1dd5ce94973e8c71a7b598f0452f`
R3	`30545673eb071f33b62939f53c50dc79` `2c87b89ef61ed6cd77a3de9f69542bd2`
R4	`3f8a1a552e009a6e95f08c7e935f8595` `78e10407e7e9b7c92e4945559d82493e`
R5	`ce9b88812bbd4bb8b9e8023adc4b1295` `1718c188597c0f59c679e30f07b241b0`
R6	`521b1682ef61d4e0e89ec8e4158e4afa` `c3d2d509447bd12624f5cf9125bc83ea`
R7	`200e5e76545fd892a820b99c21b7d0f2` `8ae2277ce7fc5707a78feda95d6bd2a0`
R8	`6f4ace6054a91bd20fc5f2f3c6deafdc` `f3b868e9dccca7fadcad5faf8b2f129c`
R9	`2e1e246dc9a4c4a43010ccc2a022129b` `0d3c2ea51735498eef7b4316479fa5ea`
R10	`eb33effa4641b6a69efe76b81beb0a1f` `bec4beeed0525465913f5247bfca073a`
R11	`b48861710101d87932da6a62ea10fc65` `2d646395f3104a27308ab1eca4628cad`
R12	`3443824ce8a989047be4f6f8c3071cc4` `7aef66a535951fcf57787eb7c7d65723`
R13	`fd335359dfbe70789f5b089fe045cd17` `c0d9fff9c1c8290fb8d630076f697687`
R14	`1bbf7f886019c586e8785356b197217e` `29ddb94097f6285c420d2caccc6da207`

6.2 Exact native commands

The five graph commands below run once from the stated working directory. The first creates a linear process specification (LPS) using regular linearisation, followed by constant and sum elimination. The next explores the complete reachable graph, and the last minimizes it modulo strong bisimulation [2, 3, 4, 5, 6]. Linearisation and equivalence are developed in the book [1, Chs. 2, 6 and 10].

Listing 16: Shared graph generation and reduction.

```
mcrl22lps --lin-method=regular ../restaurant-conveyor-delivery-system_spec.mcrl2 model.lps
lpsconstelm model.lps model.const.lps
lpssumelm model.const.lps model.ready.lps
lps2lts --cached --rewriter=jittyc --verbose model.ready.lps complete.lts
ltsconvert --equivalence=bisim complete.lts reduced.lts
```

For R1 the four property commands are exactly:

Listing 17: Whole-formula native verification of R1.

```
lts2pbes --verbose --preprocess-modal-operators --formula=../properties/R1.mcf reduced.lts R1.pbes
pbesrewr --verbose --pbes-rewriter=quantifier-one-point R1.pbes R1.one.pbes
pbesparelm --verbose R1.one.pbes R1.optimized.pbes
pbessolve --verbose --rewriter=jittyc --solve-strategy=0 R1.optimized.pbes
```

They are repeated sequentially for R2 through R14, changing only the requirement number in the formula path and output filenames. The interrupted original R12 attempt remains in that batch's records; the revised R12 repeats these same four commands separately. For a clean rerun of the selected formulas, the following Bash block gives the sequence using the graph just generated. Each `if` skips dependent stages after a tool error, then continues with the next property. A solver's `false` is a completed result and is not a tool error.

Listing 18: Sequential checks of all fourteen whole formulas.

```
for n in {1..14}; do
  if lts2pbes --verbose --preprocess-modal-operators --formula=../properties/R${n}.mcf reduced.lts R${n}.
     pbes &&
      pbesrewr --verbose --pbes-rewriter=quantifier-one-point R${n}.pbes R${n}.one.pbes &&
      pbesparelm --verbose R${n}.one.pbes R${n}.optimized.pbes; then
    pbessolve --verbose --rewriter=jittyc --solve-strategy=0 R${n}.optimized.pbes
  fi
done
```

The recorded `verification/original/verify-report-native.sh` adds standard shell sequencing, `/usr/bin/time -p`, output capture and exit-code records to these commands. It contains no custom model checker. The compact evidence contains separate `commands.log` and `stages.tsv` records for the original batch and the guarded R12 check. `scripts/verify-report-native.sh` runs the same native pipeline for a fresh verification. Additional `ltsinfo complete.lts` and `ltsinfo reduced.lts` commands record graph counts separately [11]. No automatic timeout, alternate backend, historical graph, action hiding, or separate conjunct checks are used.

6.3 Why the transformations preserve the answer

Regular linearisation represents the parallel specification as one LPS. Constant elimination substitutes parameters proved constant from their initial values; sum elimination removes unused summation variables or substitutes equalities. These simplify representation rather than restricting reachable behaviour [2, 3, 4]. The model is untimed, so no physical-time property is being preserved or checked.

`lps2lts` explores all reachable states, without a trace or state-count cutoff. `-cached` caches summand-condition evaluation within this fresh exploration; it does not load a historical state graph. `jittyc` compiles the data rewriter to speed evaluation [5]. Completion of exploration is required before any result is reported.

`ltsconvert -equivalence=bisim` merges strongly bisimilar states. Such states match each other's transitions with the same observable label and bisimilar successors. Modal μ-calculus formulas cannot distinguish them, so the quotient preserves these safety, reachability, deadlock and progress checks [1, Chs. 2 and 6][6]. Multi-action labels and their data are retained. The formulas observe those labels and bound data rather than the file's internal state-label representation. No labels are hidden, and no weaker equivalence is used.

`lts2pbes` translates the initial-state model-checking problem into a parameterised Boolean equation system (PBES). Modal preprocessing introduces auxiliary fixed points into an equivalent formula; it changes the equation structure without changing the question [7]. One-point rewriting substitutes equalities in quantified PBES expressions [8]. Parameter elimination removes parameters that do

not affect the solution [9]. Finally, `pbessolve` instantiates and solves the resulting parity game. Strategy 0 disables on-the-fly solving; `jittyc` selects compiled rewriting [10]. These transformations are not assumptions about customer behaviour or scheduling.

6.4 Measurements and outcomes

Each stage's wall time includes input loading, computation and output writing. A property's total wall time spans all four commands and their shell overhead, excluding shared graph generation and reduction. Stage timings are measured by `/usr/bin/time -p`; total spans use the shell's timestamps. Timings are observations on this host and may differ from GUI runs or a later rerun.

A property's equation system can be larger than the reduced graph because its subformulas and monitor values create additional verification states. Reading, rewriting and saving these large files can therefore take substantial time before the solver prints its answer. For example, the native R6 solver generated 27,387,043 Boolean equations, compared with 2,466,516 states in the shared reduced graph; its log is linked in Table 8.

A native, completed `true` is PASS and `false` is FAIL. A tool error, resource failure, missing final answer, or interruption is INCONCLUSIVE. A failed stage prevents use of its output by later stages. No earlier Boolean result or timing is imported into this campaign. Table 8 links every requirement to its full formula and native evidence; shared graph work appears separately in Table 6.

Table 6: Fresh shared graph generation and reduction.

Graph	States	Transitions
Complete	5,915,640	17,854,674
Strong-bisimulation quotient	2,466,516	7,395,390

Table 7: Shared preprocessing wall times, separate from property checks.

Stage	Seconds	Exit code
`mcrl22lps`	0.12	0
`lpsconstelm`	0.04	0
`lpssumelm`	0.01	0
`lps2lts`	57.02	0
`ltsconvert`	32.19	0
Five-command total span	89.439	

The two additional `ltsinfo` inspections took 17.11 and 9.20 seconds and are excluded from the graph-generation and property totals. Their logs confirm the counts above. The reported suite completed all fourteen selected native checks: 14 PASS, 0 FAIL, and 0 INCONCLUSIVE. The table contains no historical outcomes. R12 uses the user-approved finite order-ID guard in its first conjunct. Its original attempt without an explicit ID guard was interrupted without an answer after 5596.490 seconds (93 minutes 16 seconds); the solver stage took 5261.74 seconds and exited 143. That attempt remains INCONCLUSIVE in the original records and is not counted as the revised check time. The separate revision reuses the fresh graph from this campaign, whose model and graph hashes match; it changes no English requirement or model behaviour. Both attempts are preserved in the evidence.

Table 8: Fresh results for complete R1–R14 formulas.

Requirement	Formula	Outcome	Total check time (s)	Evidence
R1	Listing 2	PASS	515.052	R1 native log
R2	Listing 3	PASS	79.130	R2 native log
R3	Listing 4	PASS	479.073	R3 native log
R4	Listing 5	PASS	342.973	R4 native log
R5	Listing 6	PASS	183.980	R5 native log
R6	Listing 7	PASS	1426.620	R6 native log
R7	Listing 8	PASS	1191.969	R7 native log
R8	Listing 9	PASS	1144.686	R8 native log
R9	Listing 10	PASS	157.185	R9 native log
R10	Listing 11	PASS	99.893	R10 native log
R11	Listing 12	PASS	253.523	R11 native log
R12	Listing 13	PASS	492.164	R12 native log
R13	Listing 14	PASS	321.313	R13 native log
R14	Listing 15	PASS	66.067	R14 native log

The evidence links point to the native solver log, or the last attempted stage after a tool failure. Commands, stage timings and exit codes are retained with the native solver logs in `verification/`. Large intermediate files and verbose rewriting logs are omitted from Git; the original full run remains available locally. The logs are evidence of a Boolean answer; no state-space counterexample is implied merely by a solver log.

Table 9: Per-property stage wall times in seconds.

	`lts2pbes`	`pbesrewr`	`pbesparelm`	`pbessolve`	Total span
R1	192.29	88.09	98.37	136.21	515.052
R2	33.21	11.71	11.71	22.45	79.130
R3	170.89	72.44	82.75	152.35	479.073
R4	171.87	48.53	59.42	62.48	342.973
R5	67.62	31.49	35.26	49.53	183.980
R6	670.68	183.66	210.21	361.30	1426.620
R7	634.02	145.81	173.85	237.15	1191.969
R8	499.29	172.88	182.98	289.42	1144.686
R9	69.79	24.47	24.23	38.64	157.185
R10	48.08	13.08	13.89	24.79	99.893
R11	90.39	37.43	75.18	46.70	253.523
R12	179.31	80.03	100.32	131.87	492.164
R13	131.86	55.67	56.48	77.23	321.313
R14	21.18	11.13	11.93	21.77	66.067

The stage tool times are printed to two decimal places; total spans include shell overhead and are recorded to three decimal places. A dash denotes a stage not completed or not attempted, rather than zero time.

6.5 R12: finite model and order-ID quantification

The model permits only three orders, but `OrderID = Nat` gives the formula a variable ranging over all natural numbers. A finite graph does not automatically make that quantifier finite. PBES instantiation considers formula-variable values as well as model states [10]. Schematically, omitting the table parameter, the original and revised equations contain:

$$\forall i : \mathbb{N}.\ Z(i) \qquad \text{and} \qquad \forall i : \mathbb{N}.\ \big(i \geq 3 \vee Z(i)\big).$$

Here $Z(i)$ checks the delivery obligation for order i. For an ID such as 100, no load can occur, so the original obligation is already true. That fact is buried inside recursive equations. The solver may consider further ID values without recognizing that the remaining cases are irrelevant. The explicit guard lets the rewriter discard those cases directly, leaving only IDs 0–2 as nontrivial obligations. This is a computational distinction; the requirement and model bounds are unchanged.

To investigate, we checked both complete R12 formulas on the same isolated three-state load–deliver–collect cycle, using the same native preprocessing and solver options. The guarded formula returned `true` in 3.33 seconds, including compiled-rewriter setup, and generated 28 Boolean equations. The original formula returned no answer within a 20-second diagnostic allowance and remains INCONCLUSIVE. Idle sleep was inhibited for both runs. This toy cycle is a diagnostic fixture, not an alternative verification of the restaurant system.

We used `pbespp` to print both optimized PBESs and `diff` to compare them [12]. Their only differences were the order-ID guards at the three states. This strongly implicates the quantification rather than the graph's size or laptop sleep. A finite timeout does not prove nontermination, and this test is not a full profile of the large PBES. The diagnostic record retains the tiny model, both formulas, native commands, printed equations, timings and logs. These diagnostic outcomes are separate from Table 8.

7 Discussion and conclusion

All fourteen complete formulas returned a fresh native true answer. The results support R1–R14 for the bounded model and the stated interpretations. No requirement or formula was weakened, and no model change was made to obtain these passes. The safety formulas constrain observable loading, transfer, gate, pusher, collection and emergency sequences within this model. Gate-state histories rely on the stated initial state. Snapshot monitors use the source's synchronized live-slot interface rather than reconstructing all six positions independently. R10 checks notification and clearance ordering; it does not establish physical repairs or button wiring.

The suite also observes selected internal coordination events, as distinguished in Section 3. The results establish system behaviour together with those protocol conditions; they are not an exclusively external-interface verification. The communication endpoints remain composition details, while completed reports and notifications are deliberately visible. No action hiding was used to obtain the reported results.

R11–R13 express capability: each formula asks for a successful preparation, delivery or recovery continuation after each relevant trigger. A customer or operator may act along such a path. The checks do not promise that people actually participate, or that every execution delivers every meal. R14 expresses deadlock freedom but permits endless activity; only R7's normal-stop obligation explicitly excludes endless postponement before advancement or emergency. An infinite sequence of emergencies need not provide end-to-end service.

R13 retains a deliberately stronger formula than its English wording: reset and later advancement must be possible without customer collection. The additional conditional eventual-delivery draft was not part of this campaign. No result from it, the experimental suite, or legacy S/F files is included as evidence for R1–R14.

These conclusions apply to **six positions, two tables, three total accepted orders, and atomic transfers**. They do not generalize to arbitrary restaurant sizes, repeated reuse of order numbers, mechanical stopping times, sensor failures, partly transferred plates, correct kitchen tagging, or guaranteed human participation. Those need additional modeling or separate evidence. Within this boundary, retaining the checked source, formulas, hashes, commands, timings and native solver logs makes each reported outcome open to inspection. The large graphs and PBES intermediates can be regenerated rather than stored in Git.

References

[1] J. F. Groote and M. R. Mousavi. *Modeling and Analysis of Communicating Systems*. MIT Press, 2014. ISBN 9780262027717. `https://mitpress.mit.edu/9780262027717/modeling-and-analysis-of-communicating-systems/`.

[2] mCRL2 team. *mcrl22lps — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/mcrl22lps.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[3] mCRL2 team. *lpsconstelm — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/lpsconstelm.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[4] mCRL2 team. *lpssumelm — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/lpssumelm.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[5] mCRL2 team. *lps2lts — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/lps2lts.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[6] mCRL2 team. *ltsconvert — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/ltsconvert.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[7] mCRL2 team. *lts2pbes — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/lts2pbes.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[8] mCRL2 team. *pbesrewr — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/pbesrewr.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[9] mCRL2 team. *pbesparelm — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/pbesparelm.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[10] mCRL2 team. *pbessolve — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/pbessolve.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[11] mCRL2 team. *ltsinfo — mCRL2 202607.0 user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/ltsinfo.html`. Accessed 19 October 2026; installed command help retained with the evidence.

[12] mCRL2 team. *pbespp — mCRL2 202607.0 command help and user manual*. `https://www.mcrl2.org/web/user_manual/tools/release/pbespp.html`. Installed command help consulted 19 October 2026 and retained under `verification/metadata/`.